



Rajarata University of Sri Lanka

Department of Computing

ICT1305 – Data Structures
Laboratory 03 – Linked list in C
Time: 120 Minutes

Objective:

By the end of this lab, students should be able to:

- Understand the concept and importance of linked lists in C programming.
- Define and implement node structures for linked lists.
- Perform basic operations on linked lists including insertion, deletion, and traversal.
- Implement dynamic memory allocation using **malloc()** and **free()**.
- Handle memory management and pointer operations effectively.
- Apply best practices for linked list implementation and error handling.

Part 1: Introduction to Linked Lists

- **Concept Overview**

Linked lists are linear data structures where elements are stored in nodes. Each node contains data and a pointer to the next node. Unlike arrays, linked list elements are not stored in contiguous memory locations, allowing for dynamic memory allocation.

- **Node Structure Definition**

Basic Node Structure:

```
#include <stdio.h>    // For input/output functions like printf, scanf
#include <stdlib.h>    // For memory allocation functions like malloc, free

// Define the structure for a single node
struct Node {
    int data;          // Data part - stores the actual value
    struct Node* next; // Pointer part - stores address of next node
};
```

Memory Representation:

Node: [data | next] -> [data | next] -> [data | NULL]



Advantages of Linked Lists

1. Dynamic memory allocation
2. Efficient insertion and deletion
3. No memory waste
4. Size can be determined at runtime

Disadvantages:

1. No random access (must traverse from head)
2. Extra memory overhead for storing pointers
3. Not cache friendly

Part 2: Creating Nodes and Basic Operations

- Creating a New Node

Function to create a new node:

```

// Function to create a new node with given data
struct Node* createNode(int data) {
    // Allocate memory for a new node
    // malloc returns void*, we cast it to struct Node*
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    // Check if memory allocation was successful
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return NULL; // Return NULL if allocation fails
    }

    // Initialize the new node
    newNode->data = data; // Set the data value
    newNode->next = NULL; // Initialize next pointer to NULL

    return newNode; // Return pointer to the new node
}

```

- **Display/Traverse the List**

Function to display all nodes:

```
// Function to display all nodes in the linked list
void displayList(struct Node* head) {
    // Check if the list is empty
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");
    struct Node* current = head; // Start from the head node

    // Traverse until we reach the end (NULL)
    while (current != NULL) {
        printf("%d", current->data);    // Print current node's data

        if (current->next != NULL) {    // If not the last node
            printf(" -> ");            // Print arrow for visualization
        }

        current = current->next;        // Move to the next node
    }
    printf(" -> NULL\n");              // Show end of list
}
```

- Count Nodes in List

Function to count total nodes:

```
// Function to count the total number of nodes
int countNodes(struct Node* head) {
    int count = 0; // Initialize counter
    struct Node* current = head; // Start from head

    // Traverse the entire list
    while (current != NULL) {
        count++; // Increment counter for each node
        current = current->next; // Move to next node
    }

    return count; // Return total count
}
```

Part 3: Insertion Operations

- Insert at Beginning

Function to insert node at the beginning:

```
// Function to insert a node at the beginning of the list
struct Node* insertAtBeginning(struct Node* head, int data) {
    // Create a new node
    struct Node* newNode = createNode(data);
    if (newNode == NULL) return head; // Return unchanged if creation fails

    // Link the new node to the current head
    newNode->next = head; // New node points to old head

    // Update head to point to new node
    return newNode; // New node becomes the new head
}
```

- Insert at End

Function to insert node at the end:

```

// Function to insert a node at the end of the list
struct Node* insertAtEnd(struct Node* head, int data) {
    // Create new node
    struct Node* newNode = createNode(data);
    if (newNode == NULL) return head;

    // If list is empty, new node becomes head
    if (head == NULL) {
        return newNode;
    }

    // Traverse to find the last node
    struct Node* current = head;
    while (current->next != NULL) {    // Find node whose next is NULL
        current = current->next;
    }

    // Link the last node to new node
    current->next = newNode;           // Last node now points to new node

    return head;                      // Head remains unchanged
}

```

- **Insert at Specific Position**

Function to insert node at given position:

```

// Function to insert node at a specific position (0-based indexing)
struct Node* insertAtPosition(struct Node* head, int data, int position) {
    // If position is 0, insert at beginning
    if (position == 0) {
        return insertAtBeginning(head, data);
    }

    // Create new node
    struct Node* newNode = createNode(data);
    if (newNode == NULL) return head;

    // Traverse to position-1 (the node before insertion point)
    struct Node* current = head;
    for (int i = 0; i < position - 1; i++) {
        if (current == NULL) {           // Position is beyond list length
            printf("Position out of bounds!\n");
            free(newNode);               // Free the created node
            return head;
        }
        current = current->next;
    }

    // Insert the new node
    newNode->next = current->next;       // New node points to next node
    current->next = newNode;            // Previous node points to new node

    return head;
}

```

Part 4: Deletion Operations

- Delete First Node

Function to delete the first node:


```
// Function to delete the first node
struct Node* deleteFirst(struct Node* head) {
    // Check if list is empty
    if (head == NULL) {
        printf("List is empty, nothing to delete.\n");
        return NULL;
    }

    // Store the node to be deleted
    struct Node* nodeToDelete = head;

    // Update head to point to second node
    head = head->next;

    // Free the memory of deleted node
    free(nodeToDelete);

    printf("First node deleted successfully.\n");
    return head; // Return new head
}
```

- **Delete Last Node**

Function to delete the last node:

```

// Function to delete the last node
struct Node* deleteLast(struct Node* head) {
    // Case 1: Empty list
    if (head == NULL) {
        printf("List is empty, nothing to delete.\n");
        return NULL;
    }

    // Case 2: Only one node
    if (head->next == NULL) {
        free(head);
        printf("Last node deleted successfully.\n");
        return NULL; // List becomes empty
    }

    // Case 3: Multiple nodes - find second last node
    struct Node* current = head;
    while (current->next->next != NULL) { // Stop at second last node
        current = current->next;
    }

    // Delete the last node
    free(current->next); // Free last node
    current->next = NULL; // Second last becomes last

    printf("Last node deleted successfully.\n");
    return head;
}

```

- Delete Node with Specific Value

Function to delete node containing specific value:

```
// Function to delete the first occurrence of a specific value
struct Node* deleteByValue(struct Node* head, int value) {
    // Case 1: Empty List
    if (head == NULL) {
        printf("List is empty.\n");
        return NULL;
    }

    // Case 2: First node contains the value
    if (head->data == value) {
        struct Node* nodeToDelete = head;
        head = head->next;           // Update head
        free(nodeToDelete);         // Free old head
        printf("Node with value %d deleted.\n", value);
        return head;
    }

    // Case 3: Search in remaining nodes
    struct Node* current = head;
    while (current->next != NULL) {
        if (current->next->data == value) { // Found the value
            struct Node* nodeToDelete = current->next;
            current->next = current->next->next; // Skip the node to delete
            free(nodeToDelete);           // Free the node
            printf("Node with value %d deleted.\n", value);
            return head;
        }
        current = current->next;
    }

    // Value not found
    printf("Value %d not found in the list.\n", value);
    return head;
}
```

Part 5: Search Operations

- Search for a Value

Function to search for a specific value:

```
// Function to search for a specific value in the list
int searchValue(struct Node* head, int value) {
    struct Node* current = head;
    int position = 0; // Track position (0-based)

    // Traverse the list
    while (current != NULL) {
        if (current->data == value) { // Value found
            printf("Value %d found at position %d.\n", value, position);
            return position; // Return position of found value
        }
        current = current->next; // Move to next node
        position++; // Increment position counter
    }

    // Value not found
    printf("Value %d not found in the list.\n", value);
    return -1; // Return -1 to indicate not found
}
```

Part 6: Memory Management

- Free List Memory

Function to free all allocated memory:

```
// Function to free all allocated memory (cleanup)
void freeList(struct Node* head) {
    struct Node* current = head;
    struct Node* nextNode;

    // Traverse and free each node
    while (current != NULL) {
        nextNode = current->next;    // Store next node before freeing current
        free(current);              // Free current node
        current = nextNode;         // Move to next node
    }

    printf("All memory freed successfully.\n");
}
```

Important: Always call this function before program ends to prevent memory leaks.

Part 7: Complete Example Program

Main function with menu-driven interface:

```
int main() {
    Node* head = NULL;
    int choice, value, position;

    while (1) {
        printf("\n=== Linked List Operations ===\n");
        printf("1. Insert at beginning\n");
        printf("2. Insert at end\n");
        printf("3. Insert at position\n");
        printf("4. Delete first node\n");
        printf("5. Delete last node\n");
        printf("6. Delete by value\n");
        printf("7. Search value\n");
        printf("8. Display list\n");
        printf("9. Count nodes\n");
        printf("10. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                head = insertAtBeginning(head, value);
                break;

            case 2:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                head = insertAtEnd(head, value);
                break;

            case 3:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                printf("Enter position: ");
                scanf("%d", &position);
                head = insertAtPosition(head, value, position);
                break;

            case 4:
                head = deleteFirst(head);
                break;

            case 5:
                head = deleteLast(head);
                break;
```

```

        case 6:
            printf("Enter value to delete: ");
            scanf("%d", &value);
            head = deleteValue(head, value);
            break;

        case 7:
            printf("Enter value to search: ");
            scanf("%d", &value);
            searchValue(head, value);
            break;

        case 8:
            displayList(head);
            break;

        case 9:
            printf("Number of nodes: %d\n", countNodes(head));
            break;

        case 10:
            freeList(head);
            printf("Program terminated!\n");
            exit(0);

        default:
            printf("Invalid choice!\n");
    }
}

return 0;
}

```

Key Concepts

Memory Management:

- Always use **malloc()** to allocate memory for new nodes
- Check if **malloc()** returns **NULL** (allocation failure)
- Use **free()** to deallocate memory when deleting nodes
- Free all nodes before program termination

Pointer Handling:

- Initialize pointers to **NULL**
- Always check for **NULL** before dereferencing pointers
- Use -> operator to access structure members through pointers

Summary Points for Students.

- Linked lists provide dynamic memory allocation and efficient insertion/deletion operations.
- Each node contains data and a pointer to the next node.
- Always handle memory allocation failures and free allocated memory.
- Use proper pointer handling techniques to avoid segmentation faults.
- Always include necessary header files (<stdio.h> and <stdlib.h>).

Conclusion

In this session, we explored linked lists in C for dynamic data storage and manipulation. Students learned how to create nodes, perform insertion and deletion operations, search for values, and manage memory effectively. Proper understanding of pointers and memory management is crucial for working with linked lists.

-End-